

CAHIER DES CHARGES

Construction Project & Site Management

Application Mobile & Web de Gestion de Chantier

Équipe de développement

KABORE Pauline

GUSSOU Ali

Ouedraogo Moumouni

Version 1.0

January 27, 2026

Contents

1	Introduction	4
1.1	Contexte du projet	4
1.2	Objectifs	4
1.3	Périmètre	4
2	Description fonctionnelle	6
2.1	Vue d'ensemble	6
2.1.1	Flux de travail	6
2.2	Utilisateurs et rôles	6
2.3	Modules fonctionnels	7
2.3.1	Gestion des chantiers	7
2.3.2	Suivi des tâches	7
2.3.3	Gestion des équipes	8
2.3.4	Gestion des matériaux	8
2.3.5	Gestion financière	8
2.3.6	Documentation visuelle	8
2.3.7	Reporting et analyse	9
3	Spécifications techniques	10
3.1	Architecture système	10
3.1.1	Composants architecturaux	10
3.1.2	Principes architecturaux	10
3.2	Stack technologique	11
3.3	Base de données	12
3.3.1	Schéma relationnel	12
3.3.2	Relations et contraintes	13
3.4	APIs et intégrations	13
3.4.1	Endpoints principaux	13
3.4.2	Standards API	15
4	Fonctionnalités détaillées	17
4.1	Interface Web (Administration)	17
4.1.1	Dashboard principal	17
4.1.2	Gestion des chantiers	17
4.1.3	Suivi d'avancement	18
4.1.4	Gestion des ressources	18
4.1.5	Gestion financière	18
4.1.6	Rapports	19
4.1.7	Administration	19

4.2	Application Mobile (Terrain)	19
4.2.1	Authentification	19
4.2.2	Dashboard mobile	20
4.2.3	Suivi quotidien	20
4.2.4	Tâches	20
4.2.5	Présences	21
4.2.6	Photos	21
4.2.7	Dépenses	21
4.2.8	Mode hors connexion	22
5	Exigences non fonctionnelles	23
5.1	Performance	23
5.2	Disponibilité	23
5.3	Scalabilité	24
5.4	Compatibilité	24
5.5	Utilisabilité	24
5.6	Maintenabilité	25
5.7	Portabilité	25
6	Sécurité	26
6.1	Authentification	26
6.2	Autorisation	26
6.3	Protection des données	27
6.4	Confidentialité	27
6.5	Traçabilité	27
6.6	Résilience	28
7	Livrables	29
7.1	Code source	29
7.2	Documentation	29
7.3	Tests	30
7.4	Déploiement	30
7.5	Présentation	31
8	Méthodologie de développement	32
8.1	Approche Agile	32
8.1.1	Principes directeurs	32
8.1.2	Cérémonies Agile	32
8.2	Organisation de l'équipe	32
8.2.1	Membres de l'équipe	32
8.2.2	Responsabilités transversales	33
8.3	Outils de collaboration	33
8.3.1	Gestion de code	33
8.3.2	Communication	33
8.3.3	Gestion de projet	33
8.3.4	Développement et tests	34
8.4	Bonnes pratiques	34
8.4.1	Qualité du code	34
8.4.2	Gestion de version	34

8.4.3	Documentation continue	34
9	Annexes	35
9.1	Glossaire	35
9.2	Références	36
9.2.1	Documentation officielle	36
9.2.2	Standards et spécifications	36
9.2.3	Sécurité	36
9.2.4	Méthodologie	36
9.3	Diagrammes à produire	36
9.3.1	Architecture	37
9.3.2	Base de données	37
9.3.3	Processus	37
9.3.4	Déploiement	37
9.4	Conventions de codage	37
9.4.1	Backend (TypeScript/JavaScript)	37
9.4.2	Frontend (React/TypeScript)	38
9.4.3	Mobile (Dart/Flutter)	38
9.5	Checklist de qualité	38
9.5.1	Avant chaque commit	38
9.5.2	Avant chaque merge	38
9.5.3	Avant chaque release	39

Chapter 1

Introduction

1.1 Contexte du projet

Le secteur de la construction fait face à des défis majeurs en matière de gestion de chantier : manque de traçabilité, retards dans la communication, dépassements budgétaires, et difficulté à suivre l'avancement réel des travaux. L'application Construction Project & Site Management répond à ces problématiques en offrant une solution numérique complète qui centralise et digitalise l'ensemble du processus de gestion de chantier.

Cette solution vise à moderniser les pratiques traditionnelles en proposant des outils adaptés aux réalités du terrain, tout en offrant une vision globale et analytique aux décideurs. Elle permet une meilleure coordination entre les équipes, une traçabilité exhaustive des opérations, et une prise de décision basée sur des données fiables et en temps réel.

1.2 Objectifs

Les objectifs principaux de ce projet sont les suivants :

- Développer une application mobile Flutter pour le suivi terrain en temps réel
- Créer une interface web React/Next.js pour la gestion et le pilotage
- Centraliser toutes les données de chantier dans un système sécurisé
- Permettre le fonctionnement hors connexion sur le terrain
- Améliorer la communication entre le terrain et le bureau
- Réduire les pertes financières et optimiser les ressources
- Fournir des outils d'analyse et de reporting automatisés
- Assurer la traçabilité complète de toutes les opérations
- Faciliter la prise de décision grâce à des tableaux de bord analytiques

1.3 Périmètre

Le projet couvre les domaines suivants :

- Développement Backend : API REST Node.js avec Express ou NestJS
- Développement Frontend Web : application React/Next.js avec rendu côté serveur
- Développement Application Mobile : Flutter pour iOS et Android
- Système d'authentification JWT avec gestion des rôles et permissions
- Base de données relationnelle PostgreSQL ou MySQL
- Mode hors connexion avec synchronisation automatique
- Documentation API complète avec Swagger/OpenAPI
- Déploiement et mise en production
- Tests unitaires et d'intégration
- Documentation technique et guides utilisateurs

Chapter 2

Description fonctionnelle

2.1 Vue d'ensemble

L'application est composée de deux interfaces complémentaires qui fonctionnent en synergie :

Interface Web destinée aux chefs de projet, administrateurs et décideurs pour la configuration, le pilotage et l'analyse des chantiers. Elle offre une vue d'ensemble complète et des outils d'analyse avancés.

Application Mobile destinée aux superviseurs et chefs de chantier pour le suivi opérationnel quotidien sur le terrain. Elle est optimisée pour une utilisation rapide et intuitive dans des conditions difficiles.

2.1.1 Flux de travail

Le flux de travail typique suit les étapes suivantes :

1. Création et configuration du chantier via l'interface web
2. Affectation des ressources humaines et matérielles
3. Définition du planning et des budgets prévisionnels
4. Utilisation quotidienne de l'application mobile sur le terrain
5. Saisie des données : présences, avancement, photos, dépenses
6. Synchronisation automatique des données vers le serveur central
7. Consultation et analyse via l'interface web
8. Génération de rapports et prises de décisions
9. Ajustements et optimisations continues

2.2 Utilisateurs et rôles

Le système définit six rôles utilisateurs avec des permissions spécifiques :

Table 2.1: Rôles et permissions des utilisateurs

Rôle	Permissions	Interface
Super Admin	Accès complet au système, gestion des utilisateurs, configuration globale, tous les chantiers	Web + Mobile
Admin Entreprise	Gestion de tous les chantiers de l'entreprise, création de chantiers, gestion des équipes	Web + Mobile
Chef de Projet	Gestion d'un ou plusieurs chantiers spécifiques, consultation des rapports, validation des dépenses	Web + Mobile
Superviseur	Suivi terrain quotidien, saisie des données, consultation du planning de son chantier	Mobile
Comptable	Consultation des dépenses et rapports financiers, validation des coûts, export comptable	Web
Consultant	Lecture seule sur des chantiers spécifiques, consultation des rapports	Web

2.3 Modules fonctionnels

2.3.1 Gestion des chantiers

Ce module permet de :

- Créer et configurer les chantiers avec toutes les informations nécessaires
- Définir la structure hiérarchique : phases, lots, tâches
- Affecter les équipes et ressources humaines
- Définir les budgets prévisionnels par tâche ou par phase
- Archiver les chantiers terminés tout en conservant l'historique
- Gérer les localisations géographiques des chantiers

2.3.2 Suivi des tâches

Le suivi des tâches inclut :

- Création de tâches avec dépendances et prérequis
- Mise à jour du statut : à faire, en cours, terminé, bloqué
- Suivi du pourcentage d'avancement en temps réel
- Historique complet des modifications avec auteurs et dates
- Alertes automatiques en cas de retard ou blocage
- Attribution de priorités et échéances

2.3.3 Gestion des équipes

La gestion des équipes comprend :

- Enregistrement quotidien des présences des ouvriers
- Affectation des ouvriers par tâche et par chantier
- Suivi des heures travaillées et heures supplémentaires
- Analyse de productivité par ouvrier et par équipe
- Gestion des absences, congés et remplacements
- Base de données complète des ouvriers avec compétences

2.3.4 Gestion des matériaux

Ce module permet de :

- Maintenir un catalogue des matériaux utilisés
- Enregistrer les consommations par tâche et par date
- Suivre les stocks par chantier en temps réel
- Assurer la traçabilité des approvisionnements
- Générer des alertes de réapprovisionnement
- Calculer les coûts matériaux par projet

2.3.5 Gestion financière

La gestion financière inclut :

- Enregistrement des dépenses terrain avec catégorisation
- Association de justificatifs photographiques
- Comparaison automatique budget prévisionnel versus coûts réels
- Workflow de validation des dépenses
- Alertes en cas de dépassement budgétaire
- Export des données pour systèmes comptables

2.3.6 Documentation visuelle

Le module de documentation visuelle propose :

- Prise de photos d'avancement avec géolocalisation automatique
- Association des photos aux tâches et phases concernées
- Création d'une chronologie visuelle du chantier

- Organisation en galeries par phase, lot ou période
- Export pour intégration dans les rapports
- Compression optimisée pour le stockage et la transmission

2.3.7 Reporting et analyse

Le système de reporting offre :

- Génération automatique de rapports d'avancement
- Tableaux de bord personnalisables par rôle
- Exports multiformats : PDF, Excel, CSV
- Comparaisons multi-chantiers pour analyses transversales
- Calcul d'indicateurs de performance clés
- Visualisations graphiques avancées

Chapter 3

Spécifications techniques

3.1 Architecture système

L'application suit une architecture client-serveur moderne avec séparation claire des responsabilités selon le modèle MVC (Model-View-Controller) adapté aux applications web et mobiles contemporaines.

3.1.1 Composants architecturaux

Backend API RESTful Serveur Node.js avec Express ou NestJS assurant la logique métier et l'accès aux données

Frontend Web Application React/Next.js avec rendu côté serveur pour des performances optimales

Application Mobile Flutter pour iOS et Android avec code source partagé

Base de données PostgreSQL ou MySQL pour le stockage relationnel

Stockage fichiers Système de fichiers local ou cloud selon l'infrastructure

Authentification JWT avec refresh tokens pour une sécurité renforcée

Cache Redis optionnel pour optimiser les performances

3.1.2 Principes architecturaux

- Séparation des préoccupations : backend, frontend, mobile indépendants
- Architecture RESTful avec endpoints versionnés
- Stateless API pour une scalabilité horizontale
- Validation des données à tous les niveaux
- Gestion d'erreur centralisée et cohérente
- Logging structuré pour le monitoring

3.2 Stack technologique

Table 3.1: Technologies et outils du projet

Composant	Technologie	Justification
Backend	Node.js v18+	Performance, écosystème riche, JavaScript fullstack
Framework Backend	Express.js ou NestJS	Express : simple, flexible. NestJS : structuré, TypeScript natif
Langage Backend	TypeScript	Typage fort, maintenabilité, réduction des bugs
Authentification	JWT (jsonwebtoken)	Standard industriel, stateless, scalable
Hash mots de passe	bcrypt	Sécurité éprouvée, résistant aux attaques
Upload fichiers	Multer	Gestion efficace des uploads multipart
Validation	Joi ou class-validator	Validation robuste des données entrantes
Frontend Framework	React 18+	Écosystème mature, composants réutilisables
Meta-framework	Next.js 14+	SSR, routing, optimisations automatiques
Langage Frontend	TypeScript	Cohérence avec le backend, qualité du code
Styling	TailwindCSS	Productivité, design system cohérent
État serveur	React Query ou SWR	Cache intelligent, synchronisation automatique
HTTP client	Axios	Interceptors, gestion d'erreur, API claire
Graphiques	Recharts ou Chart.js	Visualisations interactives et responsives
Mobile Framework	Flutter 3.x	Performance native, UI riche, développement rapide
Langage Mobile	Dart 3.x	Optimisé pour Flutter, typage fort
État mobile	Provider/Riverpod/Bloc	Gestion d'état prévisible et testable
HTTP mobile	Dio	Interceptors, upload progression, timeout
Stockage local	Hive ou SQLite	Persistance offline, performances

Composant	Technologie	Justification
Base de données	PostgreSQL 14+ ou MySQL 8+	Fiabilité, performances, fonctionnalités avancées
ORM	Prisma ou TypeORM	Migration safe, typage TypeScript
Documentation API	Swagger/OpenAPI	Standard, interface interactive, génération clients
Conteneurisation	Docker	Environnements reproductibles, déploiement simplifié
Reverse Proxy	Nginx	Performance, SSL termination, load balancing
Process Manager	PM2	Redémarrage automatique, cluster mode
Tests Backend	Jest	Framework complet, mocking, couverture
Tests Mobile	Flutter Test	Intégré, widgets testing, performance

3.3 Base de données

3.3.1 Schéma relationnel

Le modèle de données comprend les tables principales suivantes :

users Utilisateurs du système avec authentification et rôles

`id, email, password_hash, first_name, last_name, role, phone, avatar, is_active, created_at, updated_at`

projects Chantiers avec informations générales et budgets

`id, name, description, location, start_date, end_date, status, budget, created_by, created_at, updated_at`

project_members Association utilisateurs-chantiers avec rôles

`id, project_id, user_id, role, assigned_at`

phases Phases des chantiers pour structuration

`id, project_id, name, description, order, start_date, end_date, created_at`

tasks Tâches détaillées avec suivi d'avancement

`id, phase_id, name, description, status, progress, priority, estimated_cost, actual_cost, assigned_to, start_date, end_date, created_at, updated_at`

workers Ouvriers affectés aux chantiers

`id, project_id, first_name, last_name, trade, phone, daily_rate, created_at`

attendances Présences quotidiennes des ouvriers

`id, worker_id, project_id, date, status, hours, recorded_by, created_at`

materials Matériaux utilisés sur les chantiers

`id, project_id, name, unit, quantity_used, unit_cost, task_id, recorded_at, recorded_by`

expenses Dépenses engagées sur le terrain

`id, project_id, task_id, category, amount, description, receipt_photo, date, recorded_by, validated_by, created_at`

progress_photos Photos d'avancement géolocalisées

`id, project_id, task_id, file_path, description, taken_at, taken_by, latitude, longitude, created_at`

reports Rapports générés automatiquement

`id, project_id, report_type, period_start, period_end, generated_by, file_path, created_at`

activity_logs Traçabilité complète des actions

`id, user_id, project_id, action, entity_type, entity_id, details, ip_address, created_at`

3.3.2 Relations et contraintes

- Clés étrangères avec contraintes d'intégrité référentielle
- Index sur les champs fréquemment interrogés
- Triggers pour l'audit automatique
- Vues matérialisées pour les rapports complexes
- Politiques de cascade pour les suppressions

3.4 APIs et intégrations

3.4.1 Endpoints principaux

L'API REST expose les endpoints suivants organisés par domaine fonctionnel :

Authentication

- POST `/api/auth/register` - Inscription nouvel utilisateur
- POST `/api/auth/login` - Connexion et obtention des tokens
- POST `/api/auth/refresh` - Rafraîchissement du token d'accès
- GET `/api/auth/me` - Récupération du profil utilisateur
- PUT `/api/auth/profile` - Mise à jour du profil

- POST /api/auth/change-password - Changement de mot de passe
- POST /api/auth/logout - Déconnexion et invalidation des tokens

Gestion des chantiers

- GET /api/projects - Liste des chantiers accessibles
- POST /api/projects - Création d'un nouveau chantier
- GET /api/projects/:id - Détails complets d'un chantier
- PUT /api/projects/:id - Modification d'un chantier
- DELETE /api/projects/:id - Archivage d'un chantier
- GET /api/projects/:id/members - Membres du chantier
- POST /api/projects/:id/members - Ajout d'un membre

Gestion des tâches

- GET /api/projects/:id/tasks - Liste des tâches d'un chantier
- POST /api/projects/:id/tasks - Création d'une tâche
- GET /api/tasks/:id - Détails d'une tâche
- PUT /api/tasks/:id - Modification complète d'une tâche
- PATCH /api/tasks/:id/status - Mise à jour du statut
- PATCH /api/tasks/:id/progress - Mise à jour de l'avancement
- DELETE /api/tasks/:id - Suppression d'une tâche

Gestion des ouvriers et présences

- GET /api/projects/:id/workers - Liste des ouvriers d'un chantier
- POST /api/projects/:id/workers - Ajout d'un ouvrier
- PUT /api/workers/:id - Modification d'un ouvrier
- POST /api/attendances - Enregistrement de présences
- GET /api/attendances - Consultation des présences avec filtres
- PUT /api/attendances/:id - Correction d'une présence

Gestion des dépenses

- POST /api/expenses - Enregistrement d'une dépense
- GET /api/projects/:id/expenses - Liste des dépenses d'un chantier
- GET /api/expenses/:id - Détails d'une dépense
- PUT /api/expenses/:id - Modification d'une dépense

- PATCH /api/expenses/:id/validate - Validation d'une dépense
- DELETE /api/expenses/:id - Suppression d'une dépense

Gestion des photos

- POST /api/photos - Upload d'une photo d'avancement
- GET /api/projects/:id/photos - Liste des photos d'un chantier
- GET /api/tasks/:id/photos - Photos associées à une tâche
- GET /api/photos/:id - Téléchargement d'une photo
- PUT /api/photos/:id - Mise à jour des métadonnées
- DELETE /api/photos/:id - Suppression d'une photo

Gestion des matériaux

- POST /api/materials - Enregistrement d'utilisation de matériau
- GET /api/projects/:id/materials - Liste des matériaux utilisés
- GET /api/materials/:id - Détails d'un enregistrement
- PUT /api/materials/:id - Correction d'un enregistrement
- DELETE /api/materials/:id - Suppression d'un enregistrement

Rapports et analyses

- GET /api/projects/:id/reports/:type - Génération d'un rapport
- GET /api/projects/:id/dashboard - Données du tableau de bord
- GET /api/projects/:id/statistics - Statistiques du chantier
- GET /api/reports/history - Historique des rapports générés

Synchronisation

- POST /api/sync - Synchronisation bulk pour mode offline
- GET /api/sync/status - Statut de synchronisation
- GET /api/sync/pending - Données en attente de sync

3.4.2 Standards API

- Versioning dans l'URL : /api/v1/...
- Codes HTTP standards : 200, 201, 400, 401, 403, 404, 500
- Format de réponse JSON cohérent
- Pagination pour les listes : query params `page`, `limit`

- Filtrage et tri : query params flexibles
- Messages d'erreur structurés et explicites

Chapter 4

Fonctionnalités détaillées

4.1 Interface Web (Administration)

4.1.1 Dashboard principal

Le tableau de bord principal offre une vue d'ensemble complète :

- Synthèse de tous les chantiers actifs avec statuts visuels
- Indicateurs clés de performance : budget global, avancement moyen, retards
- Graphiques dynamiques : évolution temporelle, répartition des coûts
- Alertes et notifications importantes nécessitant une action
- Accès rapide aux fonctions les plus utilisées
- Comparaisons période actuelle versus précédente
- Vue consolidée multi-chantiers pour analyses stratégiques

4.1.2 Gestion des chantiers

La gestion des chantiers comprend :

- Formulaire complet de création avec validation étape par étape
- Configuration de la structure hiérarchique : phases, lots, tâches
- Définition détaillée du budget prévisionnel par poste
- Affectation des membres de l'équipe avec rôles spécifiques
- Planification visuelle type Gantt avec dépendances
- Carte géographique interactive des chantiers
- Import/export de données de planification
- Duplication de chantiers similaires pour gain de temps

4.1.3 Suivi d'avancement

Le module de suivi offre :

- Visualisation comparative planning prévu versus réalisé
- Détail de l'avancement par phase, lot et tâche avec code couleur
- Chronologie visuelle illustrée par photos datées
- Comparaison en temps réel budget/dépenses avec écarts
- Identification automatique des blocages et retards critiques
- Prévisions d'achèvement basées sur la vitesse actuelle
- Indicateurs d'alerte précoce pour intervention proactive

4.1.4 Gestion des ressources

La gestion des ressources humaines inclut :

- Base de données complète des ouvriers avec compétences
- Historique détaillé des présences et absences
- Analyse de productivité individuelle et collective
- Gestion des taux journaliers et coûts de main d'œuvre
- Planification des affectations multi-chantiers
- Identification des surcharges ou sous-utilisations
- Export pour systèmes de paie et comptabilité

4.1.5 Gestion financière

Le module financier propose :

- Tableau exhaustif de toutes les dépenses avec filtres avancés
- Workflow de validation/rejet avec commentaires
- Analyse par catégorie et par période
- Rapprochement avec devis et factures
- Export comptable aux formats standards
- Alertes automatiques de dépassement budgétaire
- Prévisions de dépenses basées sur l'historique
- Dashboard financier temps réel par chantier

4.1.6 Rapports

Le système de rapports offre :

- Générateur de rapports avec templates personnalisables
- Export PDF professionnel avec logo et mise en page soignée
- Export Excel pour analyses approfondies
- Rapports périodiques automatiques envoyés par email
- Comparaisons inter-chantiers pour benchmarking interne
- Graphiques et visualisations pour présentations
- Planification de génération récurrente
- Archivage organisé des rapports historiques

4.1.7 Administration

Les fonctions d'administration comprennent :

- Gestion complète des utilisateurs : création, modification, désactivation
- Configuration fine des permissions par rôle
- Logs d'activité détaillés pour audit et traçabilité
- Paramètres système : catégories, unités, taux, etc.
- Sauvegarde et restauration des données
- Gestion des notifications et alertes
- Configuration des intégrations externes
- Monitoring de la santé du système

4.2 Application Mobile (Terrain)

4.2.1 Authentification

L'authentification mobile offre :

- Connexion sécurisée avec email et mot de passe
- Support de l'authentification biométrique : empreinte digitale, Face ID
- Mémorisation sécurisée de la session
- Déconnexion automatique après inactivité
- Récupération de mot de passe simplifiée
- Indicateur de force du mot de passe

4.2.2 Dashboard mobile

Le tableau de bord mobile présente :

- Liste des chantiers accessibles avec informations essentielles
- Statut de synchronisation clair et informatif
- Notifications et alertes avec badges
- Actions rapides du jour : présences, tâches prioritaires
- Statistiques résumées du chantier actif
- Accès rapide aux fonctions principales
- Mode sélection rapide de chantier

4.2.3 Suivi quotidien

Le suivi quotidien comprend :

- Checklist interactive des tâches du jour
- Enregistrement des présences en masse avec interface tactile
- Mise à jour rapide de l'avancement via slider ou saisie directe
- Prise de photos avec description et association automatique
- Signalement de problèmes avec photos et commentaires
- Enregistrement des dépenses avec justificatifs
- Consultation du planning journalier
- Commentaires et notes de suivi

4.2.4 Tâches

La gestion des tâches offre :

- Vue liste optimisée avec filtres et tri
- Détail complet de chaque tâche avec historique
- Modification intuitive du statut par boutons ou geste
- Slider de pourcentage d'avancement visuel
- Ajout de commentaires avec photos contextuelles
- Galerie de photos liées à la tâche
- Consultation de l'historique des modifications
- Notifications de tâches assignées ou échues

4.2.5 Présences

Le module de présences propose :

- Liste claire des ouvriers du chantier avec photos
- Marquage rapide présent/absent par tap ou swipe
- Enregistrement des heures travaillées si nécessaire
- Ajout rapide d'ouvrier temporaire ou externe
- Historique des présences par ouvrier consultable
- Statistiques de présence en temps réel
- Export pour validation

4.2.6 Photos

Le module photo inclut :

- Prise de photo native avec caméra du téléphone
- Compression automatique optimisée pour la transmission
- Géolocalisation automatique si autorisée
- Association facile à une tâche ou phase
- Galerie organisée par chantier, tâche ou date
- Mode hors ligne avec synchronisation différée
- Prévisualisation avant upload
- Ajout de descriptions textuelles

4.2.7 Dépenses

L'enregistrement des dépenses offre :

- Formulaire simplifié avec champs essentiels
- Catégories prédéfinies pour saisie rapide
- Photo du reçu intégrée au formulaire
- Validation automatique du montant
- Association à une tâche ou phase
- Historique des dépenses consultable
- Mode brouillon pour saisie ultérieure

4.2.8 Mode hors connexion

Le mode offline garantit :

- Stockage local sécurisé de toutes les données nécessaires
- File d'attente intelligente de synchronisation
- Indicateur visuel clair du statut de connexion
- Synchronisation automatique dès connexion disponible
- Gestion des conflits avec priorité aux données terrain
- Support complet du mode avion
- Compression des données pour économie de bande passante
- Synchronisation différentielle (seules les modifications)

Chapter 5

Exigences non fonctionnelles

5.1 Performance

Les exigences de performance sont définies comme suit :

- Temps de réponse API inférieur à 500 millisecondes pour 95% des requêtes
- Temps de chargement des pages web inférieur à 2 secondes
- Temps de lancement de l'application mobile inférieur à 3 secondes
- Support simultané de plus de 100 chantiers actifs sans dégradation
- Upload de photos inférieur à 5 secondes selon qualité de connexion
- Synchronisation efficace avec compression et delta sync
- Optimisation des requêtes base de données avec index appropriés
- Mise en cache intelligente des données fréquemment consultées

5.2 Disponibilité

La disponibilité du système doit garantir :

- Uptime minimum de 99,5% sur base mensuelle
- Sauvegardes automatiques quotidiennes avec rétention sur 30 jours
- Plan de reprise après incident documenté et testé
- Mode dégradé fonctionnel en cas de panne partielle
- Redondance des composants critiques
- Monitoring continu avec alertes automatiques
- Fenêtres de maintenance planifiées hors heures ouvrables

5.3 Scalabilité

Le système doit être scalable :

- Architecture permettant le scaling horizontal
- Support de 500 utilisateurs simultanés sans dégradation
- Croissance sans nécessité de refonte architecturale majeure
- Optimisation base de données : partitionnement, sharding si nécessaire
- Load balancing pour distribution de charge
- Mise en cache distribuée pour performances accrues
- Monitoring des métriques de performance pour anticipation

5.4 Compatibilité

Les exigences de compatibilité incluent :

- Navigateurs web : Chrome, Firefox, Safari, Edge (2 dernières versions)
- Systèmes mobiles : iOS 13 et supérieur, Android 8 et supérieur
- Design responsive : desktop, tablette, mobile avec breakpoints appropriés
- Support des écrans haute résolution et densité variable
- Fonctionnement optimal sur connexions 3G minimum
- Support des orientations portrait et paysage sur mobile

5.5 Utilisabilité

L'utilisabilité doit respecter les principes suivants :

- Interface intuitive nécessitant un apprentissage minimal
- Navigation simple avec maximum 3 clics pour toute action
- Feedback visuel immédiat pour toute interaction utilisateur
- Messages d'erreur clairs, explicites et actionnables
- Aide contextuelle et tooltips pour guidage
- Cohérence visuelle et interaction à travers l'application
- Accessibilité : contraste suffisant, taille de police adaptable
- Support des gestes tactiles standards sur mobile

5.6 Maintenabilité

La maintenabilité implique :

- Code source commenté suivant les conventions établies
- Architecture modulaire favorisant les modifications isolées
- Logs structurés avec niveaux appropriés
- Versioning de l'API avec gestion de la rétrocompatibilité
- Tests automatisés avec couverture minimale de 70%
- Documentation technique maintenue à jour
- Conventions de nommage claires et cohérentes
- Revue de code systématique avant intégration

5.7 Portabilité

Le système doit être portable :

- Déploiement containerisé via Docker pour cohérence
- Configuration externalisée via variables d'environnement
- Indépendance du système d'exploitation sous-jacent
- Support de différents fournisseurs de stockage cloud
- Export et import des données aux formats standards
- Documentation de déploiement complète et testée
- Scripts d'installation et de migration automatisés

Chapter 6

Sécurité

6.1 Authentication

Les mécanismes d'authentification comprennent :

- Hashage des mots de passe avec bcrypt, salt rounds minimum 10
- Tokens JWT signés avec secret fort et algorithme HS256 ou RS256
- Refresh tokens avec durée de vie limitée et rotation
- Limitation des tentatives de connexion : 5 maximum en 15 minutes
- Verrouillage temporaire du compte après échecs répétés
- Politique de mots de passe : longueur minimum 8 caractères, complexité
- Double authentification optionnelle pour comptes sensibles
- Expiration des sessions après inactivité prolongée

6.2 Autorisation

Le contrôle d'accès implémente :

- Contrôle d'accès basé sur les rôles strictement appliqué
- Vérification des permissions à chaque requête API
- Isolation des données par chantier : accès limité aux chantiers autorisés
- Validation des droits au niveau middleware
- Principe du moindre privilège systématiquement appliqué
- Logs des tentatives d'accès non autorisées
- Révocation immédiate des permissions lors de changements de rôle

6.3 Protection des données

La protection des données est assurée par :

- Chiffrement HTTPS/TLS 1.3 obligatoire pour toute communication
- Chiffrement des données sensibles au repos en base de données
- Validation et sanitisation systématique de toutes les entrées
- Protection contre injections SQL via requêtes paramétrées
- Protection XSS : échappement des données avant affichage
- Protection CSRF via tokens pour requêtes modifiantes
- Upload de fichiers sécurisé : validation type MIME, taille, contenu
- Séparation des environnements : développement, staging, production

6.4 Confidentialité

Le respect de la confidentialité implique :

- Conformité RGPD : consentement, transparence, droits utilisateurs
- Politique de confidentialité claire et accessible
- Consentement explicite pour collecte de données personnelles
- Droit à l'oubli : suppression complète des données sur demande
- Export des données personnelles pour portabilité
- Anonymisation des logs et données statistiques
- Limitation de la durée de conservation des données
- Notification en cas de violation de données

6.5 Traçabilité

La traçabilité est garantie par :

- Logs exhaustifs de toutes les actions critiques
- Historique immuable des modifications avec auteur et timestamp
- Enregistrement de l'adresse IP pour actions sensibles
- Audit trail complet pour investigations
- Conservation des logs minimum 2 ans
- Horodatage précis avec fuseau horaire
- Identification unique de chaque transaction
- Alertes sur activités suspectes ou anomalies

6.6 Résilience

La résilience du système repose sur :

- Validation des données côté client ET serveur
- Gestion d'erreur gracieuse sans exposition d'informations sensibles
- Rate limiting API : limitation du nombre de requêtes par utilisateur
- Protection DDoS au niveau infrastructure
- Backups chiffrés avec test de restauration régulier
- Plan de continuité d'activité documenté
- Isolation des erreurs pour éviter les cascades
- Timeout appropriés pour toutes les opérations réseau

Chapter 7

Livrables

7.1 Code source

Le code source livré comprend :

- Repository GitHub organisé avec structure claire
- Backend : API Node.js complète avec tous les modules
- Frontend Web : application React/Next.js production-ready
- Mobile : application Flutter pour iOS et Android
- Scripts de migration de base de données versionnés
- Configuration Docker et docker-compose pour déploiement
- Fichiers d'exemple de configuration : .env.example
- README détaillé par module avec instructions
- Fichiers .gitignore appropriés
- Licence et informations légales

7.2 Documentation

La documentation technique inclut :

- Documentation API complète Swagger/OpenAPI interactive
- Guide d'installation et de déploiement étape par étape
- Documentation d'architecture : diagrammes, schémas, explications
- Guide utilisateur interface web avec captures d'écran
- Guide utilisateur application mobile avec tutoriel
- Diagrammes : architecture système, modèle de données, flux de travail
- Changelog détaillé et release notes

- Documentation des variables d'environnement
- Guide de contribution pour développeurs futurs
- FAQ et résolution de problèmes courants

7.3 Tests

Les tests livrés comprennent :

- Suite de tests unitaires backend avec Jest
- Tests d'intégration API couvrant scénarios principaux
- Tests unitaires et widgets mobile avec Flutter Test
- Rapport de couverture de code avec seuils atteints
- Tests manuels documentés : cas de test, résultats attendus
- Tests de non-régression pour fonctionnalités critiques
- Tests de performance avec résultats benchmarks
- Configuration CI pour exécution automatique des tests

7.4 Déploiement

Le déploiement livré inclut :

- Application backend déployée sur environnement production
- Application web déployée et accessible via URL
- Applications mobiles : fichier APK Android, package IPA iOS
- Base de données configurée avec données initiales
- Certificats SSL configurés pour communications sécurisées
- Nom de domaine configuré avec DNS approprié
- Documentation complète de déploiement
- Scripts de déploiement automatisé
- Configuration de monitoring et alertes
- Plan de rollback en cas de problème

7.5 Présentation

Les éléments de présentation comprennent :

- Vidéo de démonstration complète durée 5 à 10 minutes
- Présentation PowerPoint ou équivalent pour soutenance
- Scénarios de démonstration documentés et testés
- Jeu de données de test réaliste et complet
- Guide de démonstration pour reproductibilité
- Supports visuels : captures d'écran, schémas

Chapter 8

Méthodologie de développement

8.1 Approche Agile

Le projet sera développé en suivant une méthodologie Agile adaptée au contexte académique et aux contraintes de l'équipe. Cette approche favorise l'itération rapide, la collaboration continue et l'adaptation aux changements.

8.1.1 Principes directeurs

- Développement itératif et incrémental par sprints
- Communication constante au sein de l'équipe
- Démonstrations régulières des fonctionnalités développées
- Adaptation rapide aux retours et changements
- Priorisation des fonctionnalités à forte valeur ajoutée
- Focus sur la qualité et la maintenabilité du code

8.1.2 Cérémonies Agile

Planification de sprint En début de sprint pour définir les objectifs et répartir les tâches entre les membres de l'équipe

Stand-up quotidien Réunion courte pour synchronisation : avancement, blocages, plan du jour

Revue de sprint Démonstration des fonctionnalités développées et collecte de feedback

Rétrospective Analyse du sprint écoulé pour amélioration continue

8.2 Organisation de l'équipe

8.2.1 Membres de l'équipe

L'équipe de développement est composée de trois membres avec responsabilités partagées :

KABORE Pauline Développement Backend et Base de données

GUISSOU Ali Développement Frontend Web et intégration

OUEDRAOGO Moumouni Développement Mobile et synchronisation

8.2.2 Responsabilités transversales

Bien que chaque membre ait un domaine de spécialisation, certaines responsabilités sont partagées :

- Tests : chacun teste son propre code et participe aux tests d'intégration
- Documentation : chacun documente son travail au fur et à mesure
- Revue de code : validation croisée pour qualité et apprentissage
- Déploiement : participation collective aux opérations de mise en production
- Design : décisions architecturales prises en équipe

8.3 Outils de collaboration

8.3.1 Gestion de code

GitHub Hébergement du code source avec branches par fonctionnalité, pull requests pour revue, et issues pour suivi des bugs et améliorations

Git Flow Stratégie de branches : main (production), develop (intégration), feature/* (nouvelles fonctionnalités)

8.3.2 Communication

Discord/Slack Canal de communication quotidien pour discussions rapides, partage de ressources et coordination

Réunions vidéo Visioconférences régulières pour planification et synchronisation d'équipe

8.3.3 Gestion de projet

Trello/Jira Tableaux Kanban pour visualisation du flux de travail : To Do, In Progress, Review, Done

Google Workspace Documents partagés pour documentation collaborative, comptes-rendus de réunions

8.3.4 Développement et tests

Visual Studio Code Éditeur de code principal avec extensions appropriées

Postman Tests et documentation des API REST

Docker Environnements de développement cohérents pour tous les membres

Figma Maquettes et prototypes d'interfaces utilisateur

8.4 Bonnes pratiques

8.4.1 Qualité du code

- Respect des conventions de codage établies pour chaque langage
- Commentaires explicatifs pour logique complexe
- Nommage clair et significatif des variables, fonctions, classes
- Refactoring régulier pour maintenir la dette technique basse
- Revue de code systématique avant fusion dans develop

8.4.2 Gestion de version

- Messages de commit descriptifs suivant convention : type(scope): description
- Commits atomiques : un commit = une modification logique
- Branches de fonctionnalité : feature/nom-fonctionnalite
- Pas de commit direct sur main ou develop
- Tags pour marquer les versions et releases

8.4.3 Documentation continue

- Documentation du code : commentaires, JSDoc, docstrings
- README à jour pour chaque module
- Changelog maintenu à jour avec chaque modification significative
- Documentation des décisions architecturales importantes
- Guides d'installation et de déploiement testés régulièrement

Chapter 9

Annexes

9.1 Glossaire

API Application Programming Interface - Interface de programmation permettant la communication entre applications

JWT JSON Web Token - Standard ouvert pour la création de tokens d'accès sécurisés

RBAC Role-Based Access Control - Contrôle d'accès basé sur les rôles des utilisateurs

REST Representational State Transfer - Style d'architecture pour services web utilisant HTTP

CRUD Create, Read, Update, Delete - Opérations de base sur les données

ORM Object-Relational Mapping - Technique de conversion entre systèmes de types incompatibles

SSR Server-Side Rendering - Rendu côté serveur pour améliorer performances et SEO

PWA Progressive Web App - Application web utilisant des technologies modernes pour expérience native

CI/CD Continuous Integration/Continuous Deployment - Pratiques d'intégration et déploiement continus

HTTPS Hypertext Transfer Protocol Secure - Version sécurisée du protocole HTTP

TLS Transport Layer Security - Protocole cryptographique pour communications sécurisées

CSRF Cross-Site Request Forgery - Type d'attaque exploitant la confiance d'un site

XSS Cross-Site Scripting - Faille de sécurité permettant injection de code malveillant

SQL Structured Query Language - Langage de requête pour bases de données relationnelles

RGPD Règlement Général sur la Protection des Données - Réglementation européenne

9.2 Références

9.2.1 Documentation officielle

- Node.js : <https://nodejs.org/docs>
- Express.js : <https://expressjs.com>
- NestJS : <https://docs.nestjs.com>
- React : <https://react.dev>
- Next.js : <https://nextjs.org/docs>
- Flutter : <https://flutter.dev/docs>
- PostgreSQL : <https://www.postgresql.org/docs>
- MySQL : <https://dev.mysql.com/doc>

9.2.2 Standards et spécifications

- JWT Introduction : <https://jwt.io/introduction>
- OpenAPI Specification : <https://swagger.io/specification>
- RESTful API Design : <https://restfulapi.net>
- HTTP Status Codes : <https://httpstatuses.com>

9.2.3 Sécurité

- OWASP Top Ten : <https://owasp.org/www-project-top-ten>
- OWASP API Security : <https://owasp.org/www-project-api-security>
- Security Best Practices : <https://cheatsheetseries.owasp.org>

9.2.4 Méthodologie

- Agile Manifesto : <https://agilemanifesto.org>
- Scrum Guide : <https://scrumguides.org>
- Git Flow : <https://nvie.com/posts/a-successful-git-branching-model>

9.3 Diagrammes à produire

Au cours du développement, les diagrammes suivants devront être créés et maintenus à jour :

9.3.1 Architecture

- Diagramme d'architecture système global
- Diagramme de composants backend
- Diagramme de composants frontend
- Architecture de l'application mobile
- Infrastructure de déploiement

9.3.2 Base de données

- Modèle Entité-Association (ERD) complet
- Diagramme de classes (si ORM utilisé)
- Relations et cardinalités détaillées

9.3.3 Processus

- Diagrammes de flux utilisateur (user flows) pour parcours principaux
- Diagrammes de séquence pour processus clés : authentification, synchronisation
- Diagrammes d'activité pour workflows complexes

9.3.4 Déploiement

- Pipeline CI/CD avec étapes de build, test, deploy
- Architecture réseau et infrastructure
- Stratégie de sauvegarde et restauration

9.4 Conventions de codage

9.4.1 Backend (TypeScript/JavaScript)

- Utiliser camelCase pour variables et fonctions
- Utiliser PascalCase pour classes et interfaces
- Préfixer interfaces avec I (ex: IUser) si convention adoptée
- Indentation : 2 espaces
- Longueur de ligne : maximum 120 caractères
- Utiliser const par défaut, let si nécessaire, jamais var
- Préférer fonctions fléchées pour callbacks
- Commentaires JSDoc pour fonctions publiques

9.4.2 Frontend (React/TypeScript)

- Composants en PascalCase
- Fichiers composants : NomComposant.tsx
- Hooks personnalisés : préfixe use (ex: useAuth)
- Props typées avec interfaces
- Déstructuration des props dans signature fonction
- Un composant par fichier sauf composants très liés

9.4.3 Mobile (Dart/Flutter)

- Fichiers en snake_case
- Classes en PascalCase
- Variables et fonctions en camelCase
- Constantes en lowerCamelCase avec const
- Widgets : préfixer avec _ si privés
- Commentaires descriptifs pour widgets complexes

9.5 Checklist de qualité

9.5.1 Avant chaque commit

- Code compilé sans erreur ni warning
- Tests unitaires passent
- Code formaté selon conventions
- Pas de code commenté inutile
- Pas de console.log ou print() oubliés
- Variables et fonctions nommées clairement

9.5.2 Avant chaque merge

- Revue de code effectuée par un pair
- Tests d'intégration passent
- Documentation mise à jour si nécessaire
- Pas de conflit avec branche cible
- Fonctionnalité testée manuellement

9.5.3 Avant chaque release

- Tous les tests automatisés passent
- Tests manuels complets effectués
- Documentation utilisateur à jour
- Changelog mis à jour
- Version taggée dans Git
- Sauvegarde effectuée avant déploiement

Document vivant - Version 1.0

Ce cahier des charges constitue la référence pour le développement du projet Construction Project & Site Management.

Il sera mis à jour au fur et à mesure de l'avancement du projet pour refléter les décisions et ajustements effectués.